FinalProject_Group7

March 7, 2026

1 Isolation Forest Model

Group 7

1.1 Model Description

Isolation Forest is an unsupervised anomaly detection algorithm based on random partitioning.

The intuitive essence is:

by repeatedly partitioning the data in a random manner, anomalous or outlying data points typically require fewer splits to be isolated on their own. The number of splits needed to isolate a data point can be used to measure its degree of abnormality. By averaging the path lengths across multiple random trees to reduce randomness, a stable anomaly score is ultimately obtained.

1.2 Application Scenarios

Isolation Forest is widely used in scenarios where rare and high-impact anomalies need to be detected in large-scale data.

Typical applications include:

1. Finance and Risk Management

- Credit card fraud detection
- Abnormal transaction monitoring
- Market manipulation detection
- Anti-Money Laundering (AML)

Reference: Núñez Delafuente et al. (2024), Mathematics (MDPI), "Ensemble Approach Using k-Partitioned Isolation Forests for the Detection of Stock Market Manipulation." https://doi.org/10.3390/math12091336

2. Cybersecurity

- Network intrusion detection
- Abnormal traffic pattern detection
- Malicious behavior identification
- Account anomaly detection

Reference: Marteau et al. (2017), arXiv, "Hybrid Isolation Forest — Application to Intrusion Detection." https://arxiv.org/abs/1705.03800

3. Industry and IoT

- Predictive maintenance
- Sensor anomaly detection
- Industrial process monitoring

Reference: Elsaid & Binbusayyis (2024), Discover Applied Sciences,
"An Optimized Isolation Forest Based Intrusion Detection System for Heterogeneous and Streaming Data in the Industrial Internet of Things (IIoT) Networks."
https://doi.org/10.1007/s42452-024-06165-w

4. Healthcare

- Rare disease pattern detection
- Abnormal vital signal monitoring
- Healthcare fraud detection

Reference: Yadav et al. (2023), IEEE ICAIIHI, "Isolation Forest Anomaly Detection in Vital Sign Monitoring for Healthcare." https://doi.org/10.1109/ICAIIHI57871.2023.10488940

5. User Behavior & Recommender Systems

- Abnormal user behavior detection
- Bot detection
- E-commerce fraud detection

Reference: Sun et al. (2016), arXiv, "Detecting Anomalous User Behavior Using an Extended Isolation Forest Algorithm." https://arxiv.org/abs/1609.06676

1.3 Key Components

1. Isolation Tree

A binary tree structure constructed based on a fully random mechanism

Binary partitioning is performed by

- randomly selecting a feature f
- randomly choosing a split threshold p within its value range

Stopping criteria:

- The node contains only one sample, or
- A predefined maximum tree depth is reached

Tree Depth Setting

To control computational complexity, the maximum tree depth is typically limited to:

$$maxdepth = \lceil \log_2 \psi \rceil$$

This depth roughly corresponds to the expected number of splits required to isolate observations in a randomly constructed binary tree.

2. Subsampling mechanism

Each tree is constructed using a small randomly selected subset of the data (The subsample size is typically set to 256 observations)

- This reduces computational complexity
- It also enhances the separability of anomalous observations

After building t trees, the probability that this observation is never selected in any tree is:

$$P_{\text{never selected}} = \left(1 - \frac{\psi}{n}\right)^t$$

In practice, the number of trees t is typically set to 100 or more. Under this setting, the probability becomes very close to zero.

Therefore, almost every observation will be included in multiple trees across the forest.

3. Path Length

The number of binary splits required to traverse from the root of the isolation tree to the leaf node where the sample is isolated

The path length of a sample x in a single isolation tree, denoted by

$$h(x)$$

Across T trees, the average path length is defined as

$$E(h(x)) = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$$

5. Normalization Factor

Through the function:

$$c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}$$

where: - ψ is sample size - $H(\psi)$ is the harmonic number

$$H(\psi) = \sum_{k=1}^{\psi} \frac{1}{k}$$

we obtain the theoretical expected number of splits required to isolate a normal observation in a random tree when the subsample size is ψ So $c(\psi)$ serves as a normalization constant, providing a baseline scale for evaluating the relative length of a path

6. Anomaly Score

Through the function:

$$s(x, \psi) = 2^{-\frac{E(h(x))}{c(\psi)}}$$

where:

- $E(h(x))$ is average path length over all trees
- $c(\psi)$ is the expected path length of a normal observation in a random tree with subsample size n

the path length is mapped to the interval (0,1)

- Yielding an interpretable measure of anomaly severity
- Ensuring that results remain comparable across different subsample sizes

Numerical Interpretation:

$$s(x) \in (0, 1)$$

- $s(x) \approx 1$, It is highly likely to be anomalous: *Short paths → easy isolation → high score*
- $s(x) \approx 0$, It is highly likely to be normal: *Long paths → difficult isolation → low score*

1.4 Algorithm Workflow

Step 1: Data Input and Parameter Settings

- Dataset $X = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^d$
- Subsample size ψ
- Number of trees T
- Contamination parameter α

Step 2: Isolation Trees Construction

For each tree $i = 1, 2, ..., t$:

- **a) Subsampling**

  Randomly draw ψ observations from X without replacement

- **b) Recursive Partitioning**

  Starting from the root node, recursively:

  - Randomly select a feature

$$f \in \{1, \dots, d\}$$

  - Randomly select a split value p such that:

$$p \sim \text{Uniform}(\min(X_f), \max(X_f))$$

  - Partition the data into:

$$Leftnode : x_f < p$$
$$Rightnode : x_f \geq p$$

- **c) Stopping Criteria**

  The recursion stops when:

  - The node contains only one observation, or
  - The tree depth reaches:

$$\lceil \log_2 \psi \rceil$$

Step 3: Path Length Computation

For an observation x, its path length in tree is:

$$h_i(x)$$

The average path length across all trees is:

$$E(h(x)) = \frac{1}{t} \sum_{i=1}^{t} h_i(x)$$

Step 4: Anomaly Score Computation

The anomaly score is defined as:

$$s(x, \psi) = 2^{-\frac{E(h(x))}{c(\psi)}}$$

Step 5: Decision Rule

Observations are ranked according to their anomaly scores

Given contamination level α:

- The top α observations with the highest scores are classified as anomalies
- The remaining observations are classified as normal

Step 6: Output

- Anomaly score for each observation
- Binary anomaly label

1.5 Available Packages

1. Scikit-learn

Class: sklearn.ensemble.IsolationForest

[1]:
```
from sklearn.ensemble import IsolationForest
```

Characteristics

- Most widely used and stable implementation
- Pure CPU implementation

- Fully integrated with sklearn ecosystem
- Production-ready and highly documented

Advantages

- Very easy to learn and use due to standardized API
- Extensive community support and learning resources
- Reliable choice for general anomaly detection tasks

When to Use

- Medium-sized datasets
- CPU-only environments
- When full sklearn pipeline compatibility is needed

2. PyOD

Class: pyod.models.iforest.IForest

[2]:
```
from pyod.models.iforest import IForest
```

Characteristics:

- Specialized outlier detection library
- Provides unified anomaly detection framework
- Built on top of sklearn
- Adds additional evaluation utilities

Advantages

- Built-in anomaly evaluation utilities
- Easy comparison with other anomaly models
- Supports unified interface for multiple detectors

When to Use

- When comparing multiple anomaly detection methods
- When evaluation tools are needed
- Research-oriented anomaly benchmarking
- Cleaner anomaly-focused API

Additional method

Method	Function
`predict_proba(`	Return probability of being an outlier

3. cuML

Class: cuml.ensemble.IsolationForest

[3]:
```
from cuml.ensemble import IsolationForest
```

Characteristics:

- GPU-accelerated implementation

- Massive speedup on large datasets
- Designed for high-performance computing

Advantages

- Significantly reduces training time on large datasets
- Enables scalable anomaly detection for massive data
- Suitable for time-sensitive industrial applications

When to Use - Very large datasets - Real-time detection systems - GPU infrastructure available - High-dimensional industrial applications

1.5.1 Common Methods

The following table summarizes the core methods commonly implemented in Isolation Forest models across different packages:

Method	Function
`fit()`	Train the model
`predict()`	Detect anomalies
`fit_predict()`	Train and detect anomalies
`decision_function()`	Compute anomaly score
`score_samples()`	Raw anomaly score

1.5.2 Key Parameters

Among them, scikit-learn provides the most comprehensive set of configurable parameters

Parameter	Meaning	sklearn	PyOD	cuML
n_estimators	Number of trees			
max_samples	Subsampling size			
contamination	Expected anomaly proportion			
max_features	Number of features used			

1.5.3 Comparison Summary

The following comparison highlights the main differences among the three implementations, each of which is suited to different usage scenarios

Package	Speed	Ease of Use	Ecosystem Integration	Best For
sklearn	Moderate	Very High	Excellent	Standard ML workflows and academic projects
PyOD	Moderate	High	Good	Anomaly detection research and model benchmarking
cuML	Very Fast	Medium	RAPIDS ecosystem	Large-scale industrial data

1.6 Final Package Choice

We choose:

Scikit-learn IsolationForest

Reasons:

- Our team is already familiar with the scikit-learn ecosystem.
- The dataset size in this project is moderate, so CPU computation is sufficient.
- GPU infrastructure is not available in our environment, making cuML less suitable.
- Scikit-learn offers the most complete and well-documented implementation of Isolation Forest.

1.7 Basic Example

[4]:
```
import numpy as np
import pandas as pd

from sklearn.ensemble import IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    precision_recall_fscore_support,
    roc_auc_score,
)

import matplotlib.pyplot as plt
```

[5]:
```
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

[6]:
```
# Inliers
# Generate 1,000 normal data points sampled from a 2D Gaussian (normal)␣
↪distribution
n_inliers = 1000
X_in = np.random.normal(loc=0.0, scale=1.0, size=(n_inliers, 2))

# Outliers
# Generate 50 anomalous points using a uniform distribution
# Values are sampled randomly between -8 and 8
n_outliers = 50
X_out = np.random.uniform(low=-8, high=8, size=(n_outliers, 2))

X = np.vstack([X_in, X_out])
# The labels are for evaluation only
```

```
y = np.hstack([np.zeros(n_inliers, dtype=int), np.ones(n_outliers, dtype=int)])␣
↪ # 0=inlier, 1=outlier

df = pd.DataFrame(X, columns=["x1", "x2"])
df["is_outlier"] = y

print(df.head())
print("\nClass balance:")
print(df["is_outlier"].value_counts(normalize=True).rename("proportion"))
```

```
         x1        x2  is_outlier
0  0.496714 -0.138264           0
1  0.647689  1.523030           0
2 -0.234153 -0.234137           0
3  1.579213  0.767435           0
4 -0.469474  0.542560           0

Class balance:
is_outlier
0    0.952381
1    0.047619
Name: proportion, dtype: float64
```

[7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    df[["x1", "x2"]].values,
    df["is_outlier"].values,
    test_size=0.30,
    random_state=RANDOM_STATE,
    stratify=df["is_outlier"].values,
)
```

[8]:
```
# Fit Isolation Forest
contamination_guess = 0.05
model = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        ("iforest", IsolationForest(
            n_estimators=300,
            contamination=contamination_guess,
            max_samples="auto",
            random_state=RANDOM_STATE,
            n_jobs=-1,
        )),
    ]
)
model.fit(X_train)
```

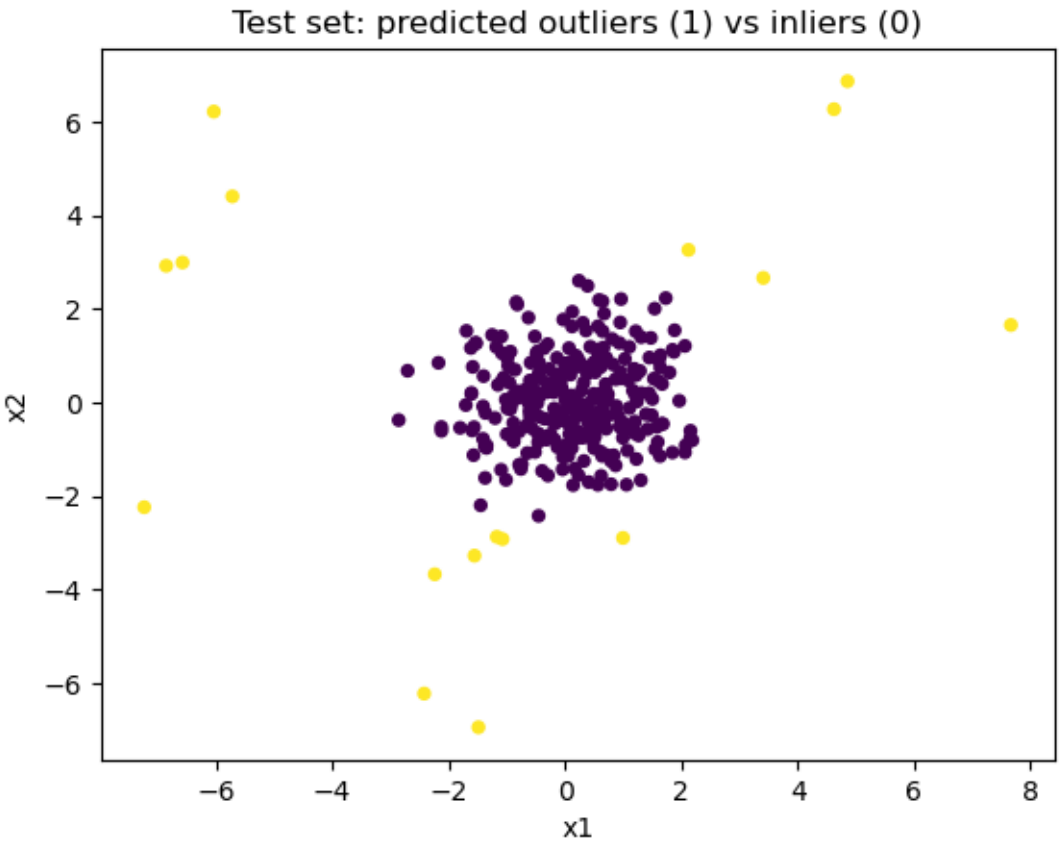
[8]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('iforest',
                 IsolationForest(contamination=0.05, n_estimators=300,
                                 n_jobs=-1, random_state=42))])
```

[9]:
```
pred_test_raw = model.predict(X_test)
y_pred_test = (pred_test_raw == -1).astype(int)  # convert to 1=outlier,␣
 ↪0=inlier

# decision_function: higher means more "normal"
# To make "outlier score" where higher = more anomalous, we invert sign.
normality_score_test = model.decision_function(X_test)
outlier_score_test = -normality_score_test
```

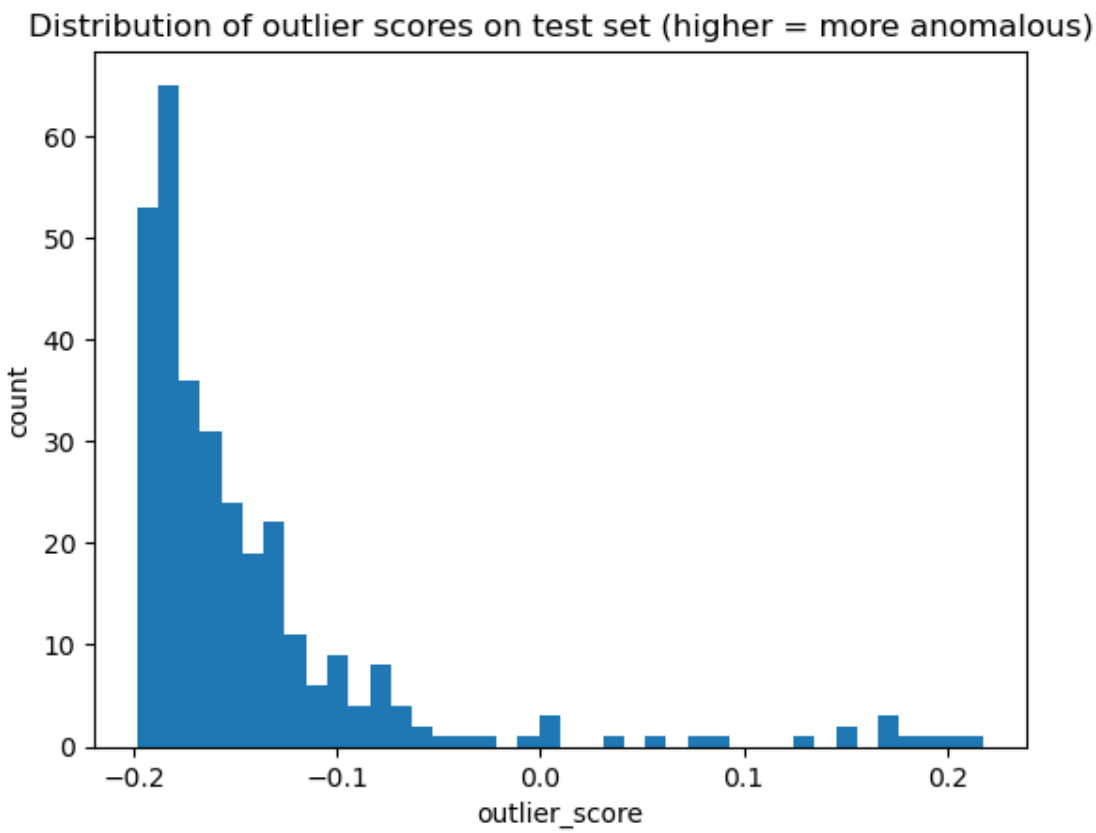
[10]:
```
# Evaluation
cm = confusion_matrix(y_test, y_pred_test)
print("\nConfusion matrix:\n", cm)

print("\nClassification report (outlier=1):")
print(classification_report(y_test, y_pred_test, digits=4))

auc = roc_auc_score(y_test, outlier_score_test)
print(f"ROC AUC: {auc:.4f}")
```

```
Confusion matrix:
 [[297   3]
 [  1  14]]

Classification report (outlier=1):
              precision    recall  f1-score   support

           0     0.9966    0.9900    0.9933       300
           1     0.8235    0.9333    0.8750        15

    accuracy                         0.9873       315
   macro avg     0.9101    0.9617    0.9342       315
weighted avg     0.9884    0.9873    0.9877       315

ROC AUC: 0.9993
```

[11]:
```
plt.figure()
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred_test, s=18)
plt.title("Test set: predicted outliers (1) vs inliers (0)")
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()
```



[12]:
```
plt.figure()
plt.hist(outlier_score_test, bins=40)
plt.title("Distribution of outlier scores on test set (higher = more␣
↪anomalous)")
plt.xlabel("outlier_score")
plt.ylabel("count")
plt.show()
```



[13]:
```
k = 10
top_idx = np.argsort(outlier_score_test)[-k:][::-1]
top_df = pd.DataFrame(X_test, columns=["x1", "x2"]).iloc[top_idx].copy()
top_df["outlier_score"] = outlier_score_test[top_idx]
top_df["true_outlier"] = y_test[top_idx]
top_df["pred_outlier"] = y_pred_test[top_idx]
print("\nTop most anomalous test points:")
print(top_df)
```

```
Top most anomalous test points:
           x1        x2  outlier_score  true_outlier  pred_outlier
277 -6.033269  6.218545       0.218253             1             1
126  4.855724  6.867649       0.206411             1             1
107  4.622920  6.269954       0.194557             1             1
40  -5.713985  4.405538       0.182924             1             1
53  -6.851587  2.921876       0.174684             1             1
9   -7.229024 -2.239193       0.174125             1             1
122 -6.571368  2.992308       0.171690             1             1
2   -2.418871 -6.224030       0.152046             1             1
208 -1.486296 -6.943842       0.152045             1             1
```

```
197  7.663698  1.655890       0.128636              1              1
```

[14]:
```python
# Prediction Example
x_new = np.array([[3.5, 3.5], [0.1, -0.1]])
pred_new_raw = model.predict(x_new)
pred_new = (pred_new_raw == -1).astype(int)
score_new = -model.decision_function(x_new)

print("New points prediction:")
for i in range(x_new.shape[0]):
    print(f"x={x_new[i]} -> pred_outlier={pred_new[i]} (1=outlier),␣
 ↪outlier_score={score_new[i]:.4f}")
```

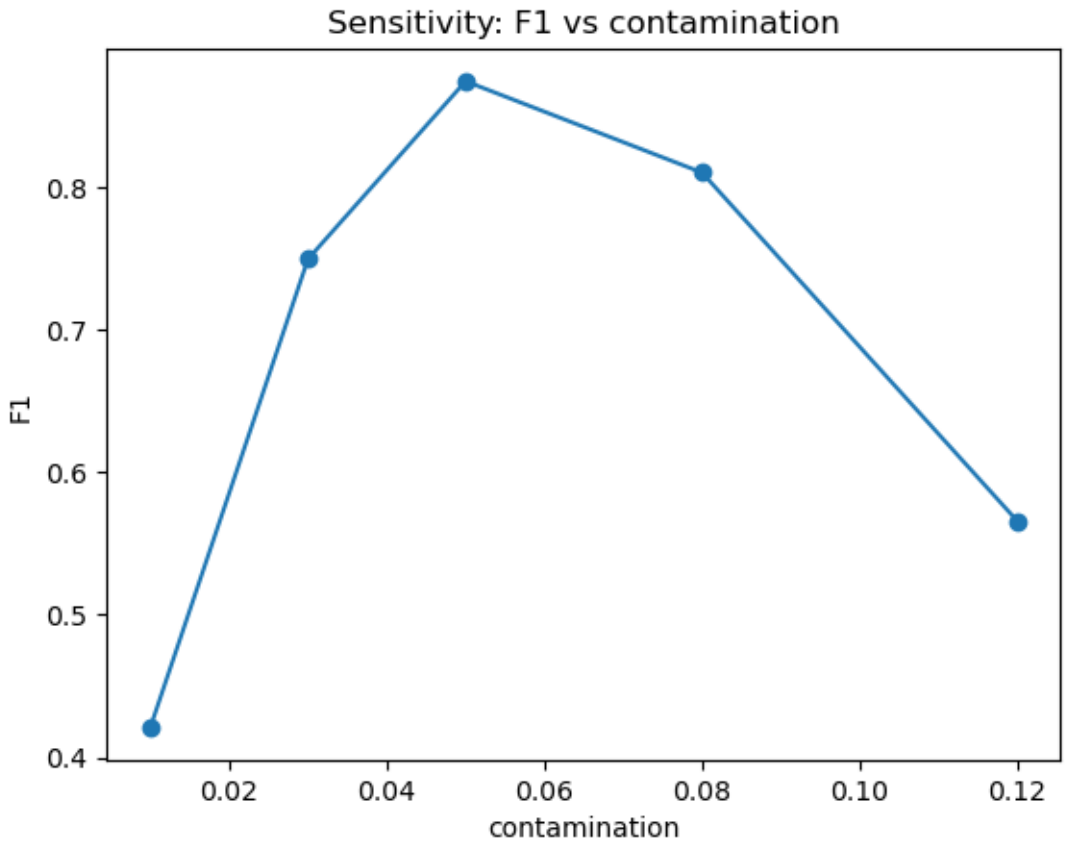
```
New points prediction:
x=[3.5 3.5] -> pred_outlier=1 (1=outlier), outlier_score=0.1107
x=[ 0.1 -0.1] -> pred_outlier=0 (1=outlier), outlier_score=-0.1972
```

[15]:
```python
# Sensitivity Analysis
# For Isolation Forest, important parameters include: contamination,␣
 ↪n_estimators, max_samples
# Helper Function
def fit_eval_iforest(
    X_train, X_test, y_test,
    contamination=0.05, n_estimators=300, max_samples="auto"
):
    pipe = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("iforest", IsolationForest(
                n_estimators=n_estimators,
                contamination=contamination,
                max_samples=max_samples,
                random_state=RANDOM_STATE,
                n_jobs=-1,
            )),
        ]
    )
    pipe.fit(X_train)

    pred_raw = pipe.predict(X_test)
    y_pred = (pred_raw == -1).astype(int)

    outlier_score = -pipe.decision_function(X_test)

    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test, y_pred, average="binary", zero_division=0
    )
    auc = roc_auc_score(y_test, outlier_score)
```

```
    return {
        "contamination": contamination,
        "n_estimators": n_estimators,
        "max_samples": max_samples,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "auc": auc,
        "pred_outlier_rate": float(y_pred.mean()),
    }
```

[16]:
```
# Contamination
contamination_grid = [0.01, 0.03, 0.05, 0.08, 0.12]
results_contam = [
    fit_eval_iforest(X_train, X_test, y_test, contamination=c)
    for c in contamination_grid
]
res_contam_df = pd.DataFrame(results_contam).sort_values("contamination")
print("Sensitivity: contamination")
display(res_contam_df)

plt.figure()
plt.plot(res_contam_df["contamination"], res_contam_df["f1"], marker="o")
plt.title("Sensitivity: F1 vs contamination")
plt.xlabel("contamination")
plt.ylabel("F1")
plt.show()
```

```
Sensitivity: contamination

   contamination  n_estimators max_samples  precision    recall        f1  \
0           0.01           300        auto   1.000000  0.266667  0.421053
1           0.03           300        auto   1.000000  0.600000  0.750000
2           0.05           300        auto   0.823529  0.933333  0.875000
3           0.08           300        auto   0.681818  1.000000  0.810811
4           0.12           300        auto   0.394737  1.000000  0.566038

        auc  pred_outlier_rate
0  0.999333           0.012698
1  0.999333           0.028571
2  0.999333           0.053968
3  0.999333           0.069841
4  0.999333           0.120635
```

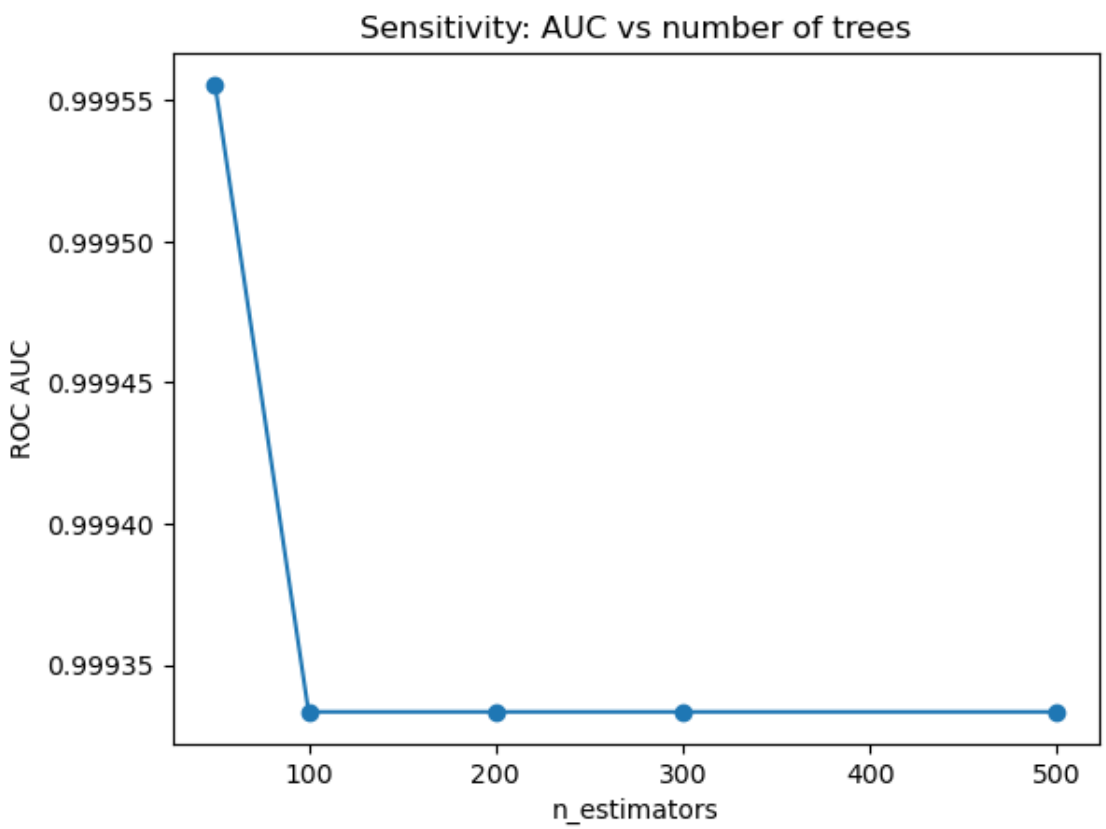


Sensitivity analysis shows that the contamination parameter changes the balance between precision and recall. If contamination is set too low, the model misses many true anomalies. If it is set too high, it flags too many normal points as anomalies. The best performance occurs when contamination is close to the true anomaly rate (around 5%). The AUC stays almost the same across settings, which means the model separates normal and anomalous points very well regardless of the threshold.

[17]:
```
# N-estimators
n_estimators_grid = [50, 100, 200, 300, 500]
results_trees = [
    fit_eval_iforest(X_train, X_test, y_test,␣
 ↪contamination=contamination_guess, n_estimators=n)
    for n in n_estimators_grid
]
res_trees_df = pd.DataFrame(results_trees).sort_values("n_estimators")
print("Sensitivity: n_estimators")
display(res_trees_df)

plt.figure()
plt.plot(res_trees_df["n_estimators"], res_trees_df["auc"], marker="o")
```

```
plt.title("Sensitivity: AUC vs number of trees")
plt.xlabel("n_estimators")
plt.ylabel("ROC AUC")
plt.show()
```

```
Sensitivity: n_estimators

   contamination  n_estimators max_samples  precision    recall        f1  \
0           0.05            50        auto   0.875000  0.933333  0.903226
1           0.05           100        auto   0.933333  0.933333  0.933333
2           0.05           200        auto   0.933333  0.933333  0.933333
3           0.05           300        auto   0.823529  0.933333  0.875000
4           0.05           500        auto   0.823529  0.933333  0.875000

        auc  pred_outlier_rate
0  0.999556           0.050794
1  0.999333           0.047619
2  0.999333           0.047619
3  0.999333           0.053968
4  0.999333           0.053968
```

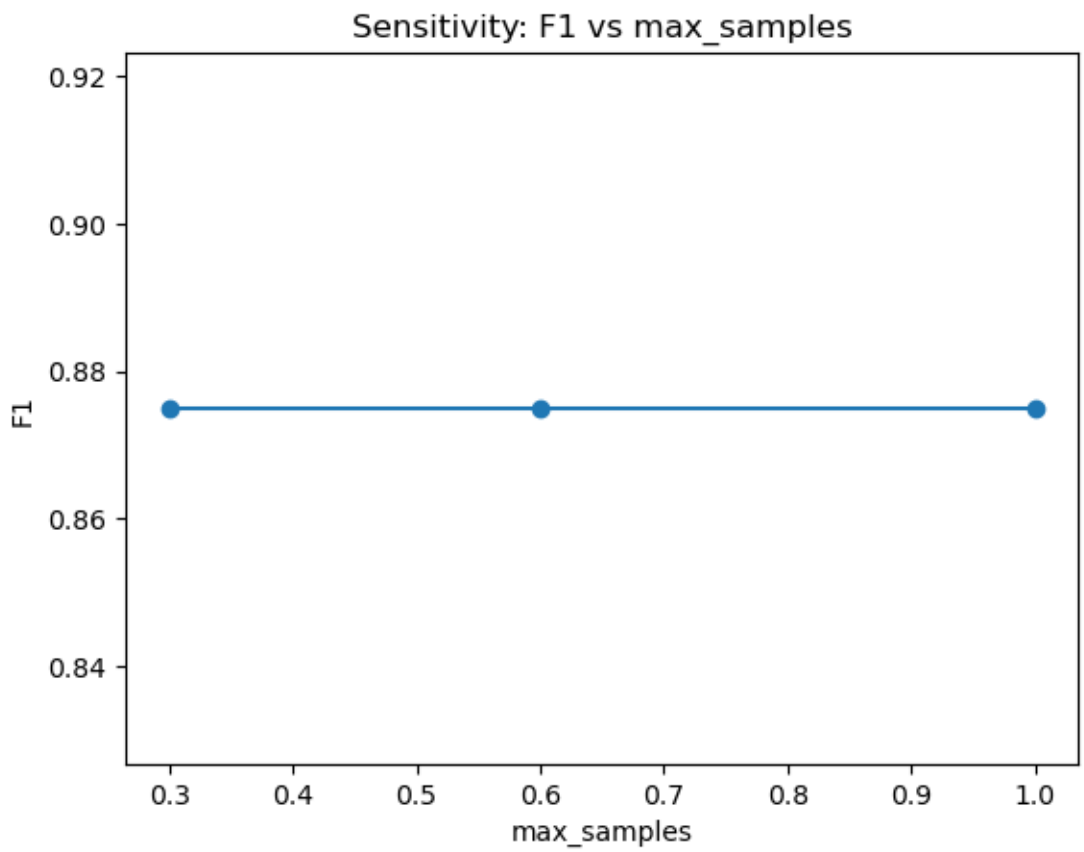


The performance is very stable. Precision, recall, F1, and AUC stay almost the same across 50 to

500 trees. This indicates that the model is not very sensitive to the number of trees.

[18]:
```
# Max samples
max_samples_grid = [0.3, 0.6, 1.0]
results_samples = [
    fit_eval_iforest(X_train, X_test, y_test,␣
 ↪contamination=contamination_guess, n_estimators=300, max_samples=ms)
    for ms in max_samples_grid
]
res_samples_df = pd.DataFrame(results_samples).sort_values("max_samples")
print("Sensitivity: max_samples")
display(res_samples_df)

plt.figure()
plt.plot(res_samples_df["max_samples"], res_samples_df["f1"], marker="o")
plt.title("Sensitivity: F1 vs max_samples")
plt.xlabel("max_samples")
plt.ylabel("F1")
plt.show()
```

```
Sensitivity: max_samples

   contamination  n_estimators  max_samples  precision    recall     f1  \
0           0.05           300          0.3   0.823529  0.933333  0.875
1           0.05           300          0.6   0.823529  0.933333  0.875
2           0.05           300          1.0   0.823529  0.933333  0.875

        auc  pred_outlier_rate
0  0.999333           0.053968
1  0.999111           0.053968
2  0.999333           0.053968
```



This output shows that changing max_samples has almost no effect on the model's performance. Precision, recall, F1, and AUC remain nearly identical across 0.3, 0.6, and 1.0. The predicted outlier rate also stays the same. This means the model is not sensitive to how many samples are used per tree in this dataset.

1.8 Real-World Application: Student Performance

[19]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import IsolationForest
from sklearn.metrics import confusion_matrix, classification_report,␣
↪roc_auc_score, precision_recall_fscore_support
```

```
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

[20]:
```
mat_path = "student/student-mat.csv"
por_path = "student/student-por.csv"

df_mat = pd.read_csv(mat_path, sep=";")
df_mat["subject"] = "mat"
df_por = pd.read_csv(por_path, sep=";")
df_por["subject"] = "por"
df = pd.concat([df_mat, df_por], axis=0, ignore_index=True)
```

[21]:
```
df.head()
```

[21]:
```
  school sex  age address famsize Pstatus  Medu  Fedu     Mjob      Fjob  …  \
0     GP   F   18       U     GT3       A     4     4  at_home   teacher  …
1     GP   F   17       U     GT3       T     1     1  at_home     other  …
2     GP   F   15       U     LE3       T     1     1  at_home     other  …
3     GP   F   15       U     GT3       T     4     2   health  services  …
4     GP   F   16       U     GT3       T     3     3    other     other  …

  freetime goout  Dalc  Walc  health absences  G1  G2  G3 subject
0        3     4     1     1       3        6   5   6   6     mat
1        3     3     1     1       3        4   5   5   6     mat
2        3     2     2     3       3       10   7   8  10     mat
3        2     2     1     1       5        2  15  14  15     mat
4        3     2     1     2       5        4   6  10  10     mat

[5 rows x 34 columns]
```

[22]:
```
df.shape
```

[22]:
```
(1044, 34)
```

[23]:
```
print("Missing values:")
missing = df.isna().sum()
missing
```

```
Missing values:
```

[23]:
```
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
```

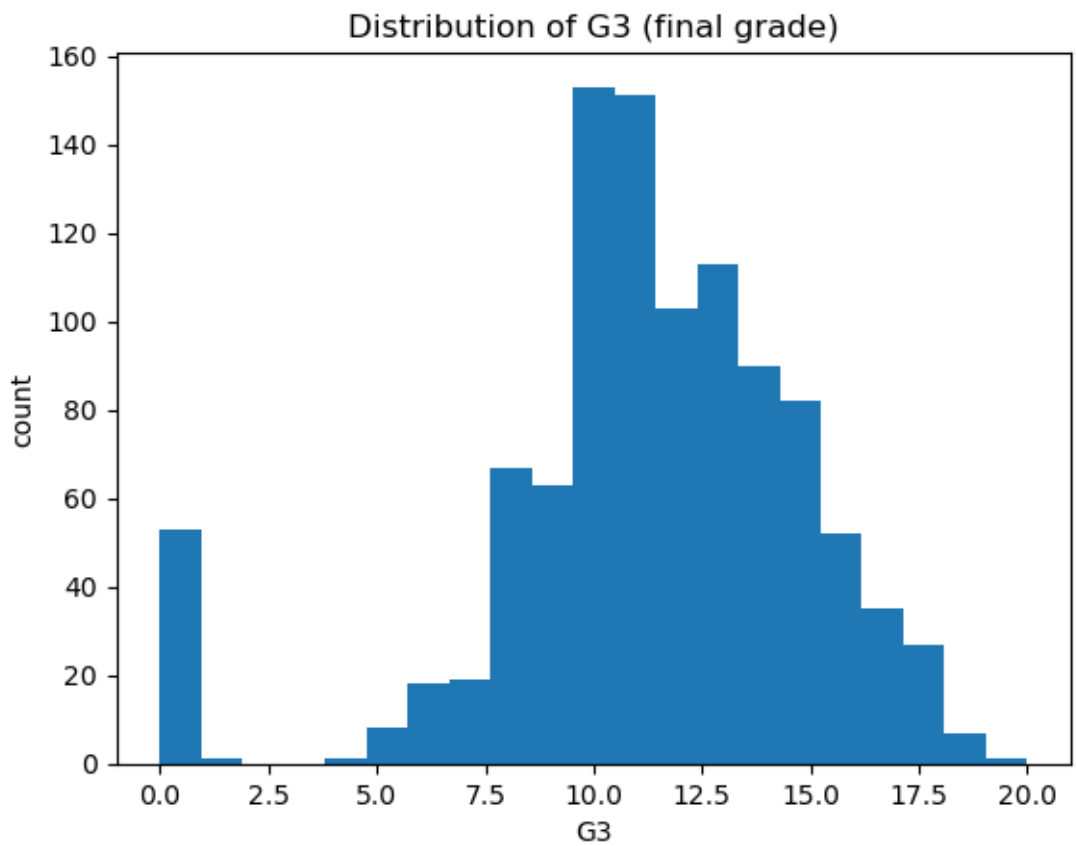
```
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
subject       0
dtype: int64
```

[24]:
```python
print("Basic stats (numeric):")
display(df.describe())
```

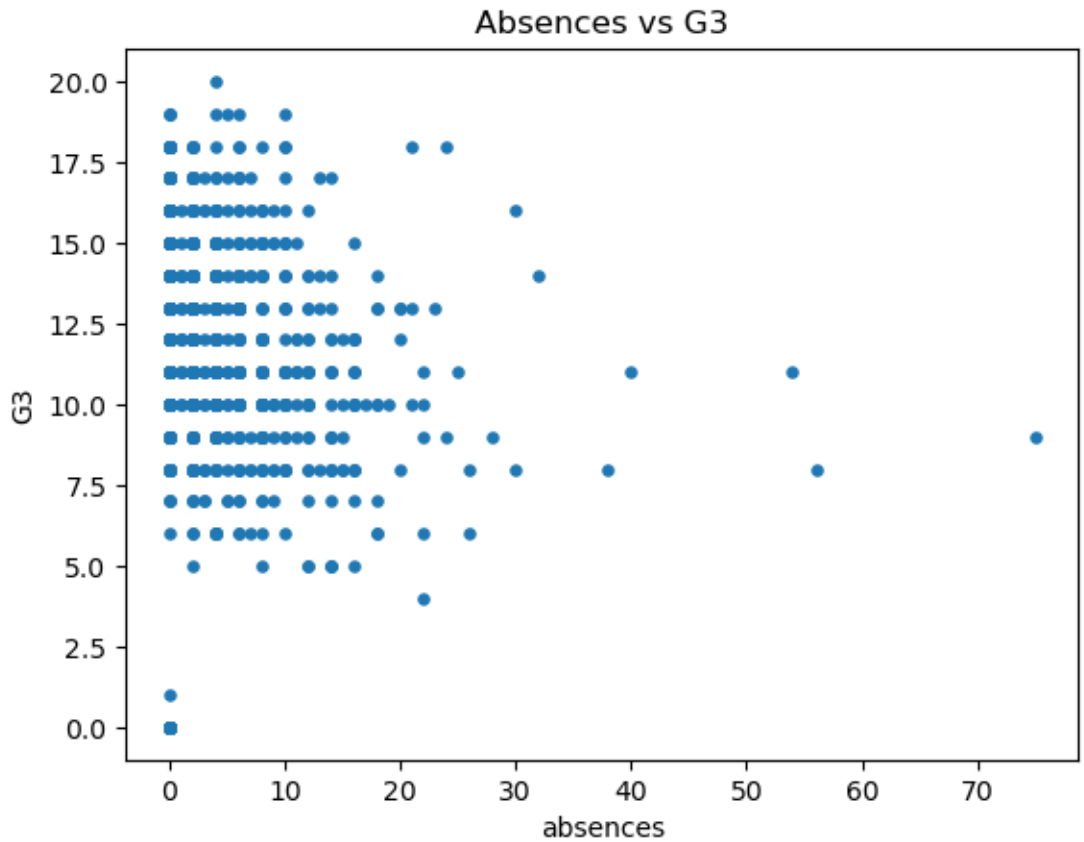
```
Basic stats (numeric):

               age         Medu         Fedu   traveltime    studytime  \
count  1044.000000  1044.000000  1044.000000  1044.000000  1044.000000
mean     16.726054     2.603448     2.387931     1.522989     1.970307
std       1.239975     1.124907     1.099938     0.731727     0.834353
min      15.000000     0.000000     0.000000     1.000000     1.000000
25%      16.000000     2.000000     1.000000     1.000000     1.000000
50%      17.000000     3.000000     2.000000     1.000000     2.000000
75%      18.000000     4.000000     3.000000     2.000000     2.000000
max      22.000000     4.000000     4.000000     4.000000     4.000000

          failures       famrel     freetime        goout         Dalc  \
count  1044.000000  1044.000000  1044.000000  1044.000000  1044.000000
mean      0.264368     3.935824     3.201149     3.156130     1.494253
std       0.656142     0.933401     1.031507     1.152575     0.911714
min       0.000000     1.000000     1.000000     1.000000     1.000000
```

```
25%       0.000000     4.000000     3.000000     2.000000     1.000000
50%       0.000000     4.000000     3.000000     3.000000     1.000000
75%       0.000000     5.000000     4.000000     4.000000     2.000000
max       3.000000     5.000000     5.000000     5.000000     5.000000

              Walc       health     absences           G1           G2  \
count  1044.000000  1044.000000  1044.000000  1044.000000  1044.000000
mean      2.284483     3.543103     4.434866    11.213602    11.246169
std       1.285105     1.424703     6.210017     2.983394     3.285071
min       1.000000     1.000000     0.000000     0.000000     0.000000
25%       1.000000     3.000000     0.000000     9.000000     9.000000
50%       2.000000     4.000000     2.000000    11.000000    11.000000
75%       3.000000     5.000000     6.000000    13.000000    13.000000
max       5.000000     5.000000    75.000000    19.000000    19.000000

                G3
count  1044.000000
mean     11.341954
std       3.864796
min       0.000000
25%      10.000000
50%      11.000000
75%      14.000000
max      20.000000
```

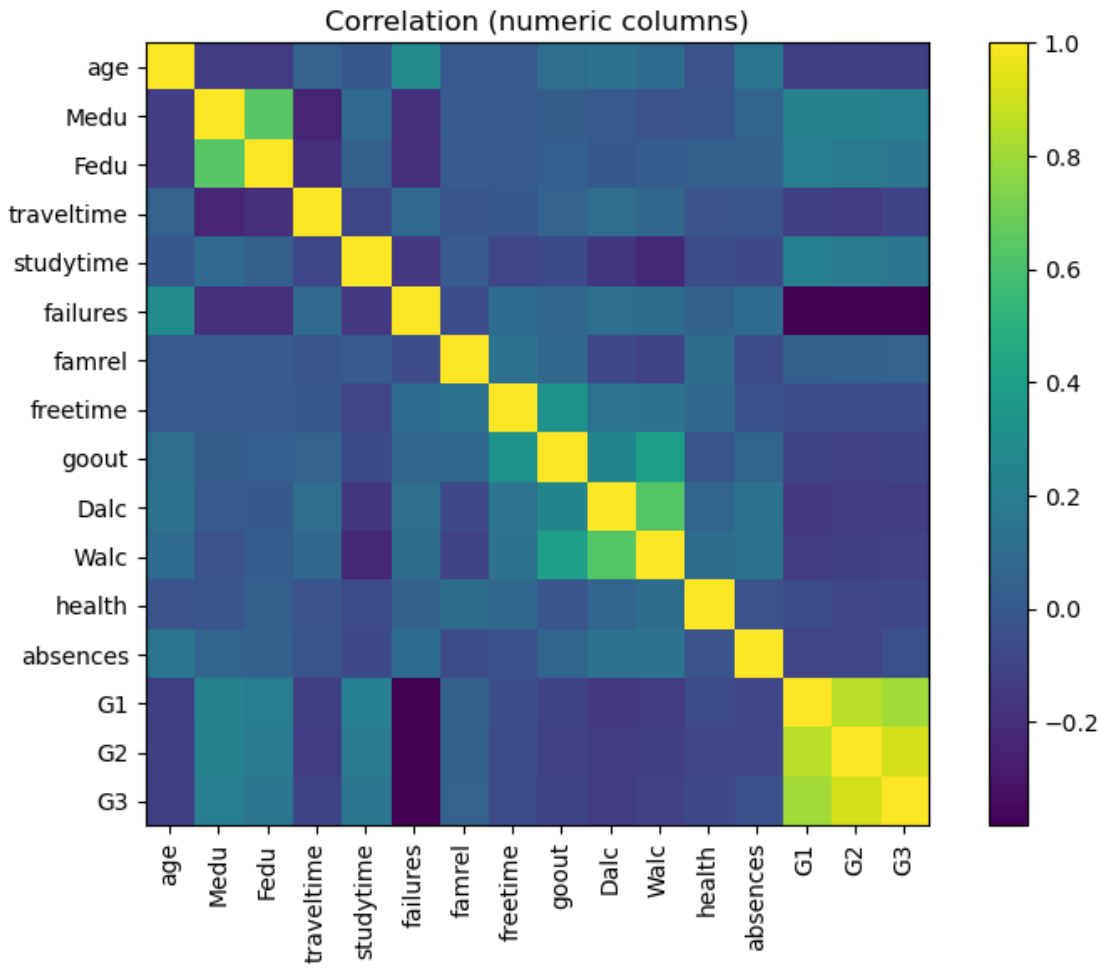
[25]:
```
plt.figure()
plt.hist(df["G3"], bins=21)
plt.title("Distribution of G3 (final grade)")
plt.xlabel("G3")
plt.ylabel("count")
plt.show()
```



[26]:
```
plt.figure()
plt.scatter(df["absences"], df["G3"], s=14)
plt.title("Absences vs G3")
plt.xlabel("absences")
plt.ylabel("G3")
plt.show()
```



[27]:
```
numeric_cols_all = df.select_dtypes(include=[np.number]).columns.tolist()
corr = df[numeric_cols_all].corr()

plt.figure(figsize=(8, 6))
plt.imshow(corr.values)
plt.title("Correlation (numeric columns)")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.index)), corr.index)
plt.colorbar()
plt.tight_layout()
plt.show()
```



[28]:
```
print("Correlation with G3 (numeric only):")
display(corr["G3"].sort_values(ascending=False))
```

```
Correlation with G3 (numeric only):

G3            1.000000
G2            0.910743
G1            0.809142
Medu          0.201472
studytime     0.161629
Fedu          0.159796
famrel        0.054461
absences     -0.045671
freetime     -0.064890
health       -0.080079
goout        -0.097877
traveltime   -0.102627
Walc         -0.115740
```

```
age          -0.125282
Dalc         -0.129642
failures     -0.383145
Name: G3, dtype: float64
```

[29]:
```
DROP_COLS = ["G1", "G2"]
df_model = df.drop(columns=DROP_COLS).copy()
df_model = df_model.drop_duplicates().reset_index(drop=True)
print("After dropping G1/G2 and exact duplicates:", df_model.shape)
```

```
After dropping G1/G2 and exact duplicates: (1044, 32)
```

[30]:
```
cat_cols = df_model.select_dtypes(include=["object"]).columns.tolist()
num_cols = df_model.select_dtypes(include=[np.number]).columns.tolist()
```

[31]:
```
X = df_model[cat_cols + num_cols].copy()

# Preprocessing:
# - numeric: standardize
# - categorical: one-hot encode
preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),␣
 ↪cat_cols),
    ],
    remainder="drop",
)
```

[32]:
```
X_train, X_test = train_test_split(
    X, test_size=0.30, random_state=RANDOM_STATE
)
```

[33]:
```
base_contamination = 0.08

pipe = Pipeline(
    steps=[
        ("prep", preprocess),
        ("iforest", IsolationForest(
            n_estimators=200,
            contamination=base_contamination,
            max_samples="auto",
            random_state=RANDOM_STATE,
            n_jobs=-1,
        ))
    ]
)
```

```
pipe.fit(X_train)
```

[33]:
```
Pipeline(steps=[('prep',
                 ColumnTransformer(transformers=[('num', StandardScaler(),
                                                  ['age', 'Medu', 'Fedu',
                                                   'traveltime', 'studytime',
                                                   'failures', 'famrel',
                                                   'freetime', 'goout', 'Dalc',
                                                   'Walc', 'health', 'absences',
                                                   'G3']),
                                                 ('cat',
OneHotEncoder(handle_unknown='ignore',
sparse_output=False),
                                                  ['school', 'sex', 'address',
                                                   'famsize', 'Pstatus', 'Mjob',
                                                   'Fjob', 'reason', 'guardian',
                                                   'schoolsup', 'famsup',
                                                   'paid', 'activities',
                                                   'nursery', 'higher',
                                                   'internet', 'romantic',
                                                   'subject'])])),
                ('iforest',
                 IsolationForest(contamination=0.08, n_estimators=200,
                                 n_jobs=-1, random_state=42))])
```

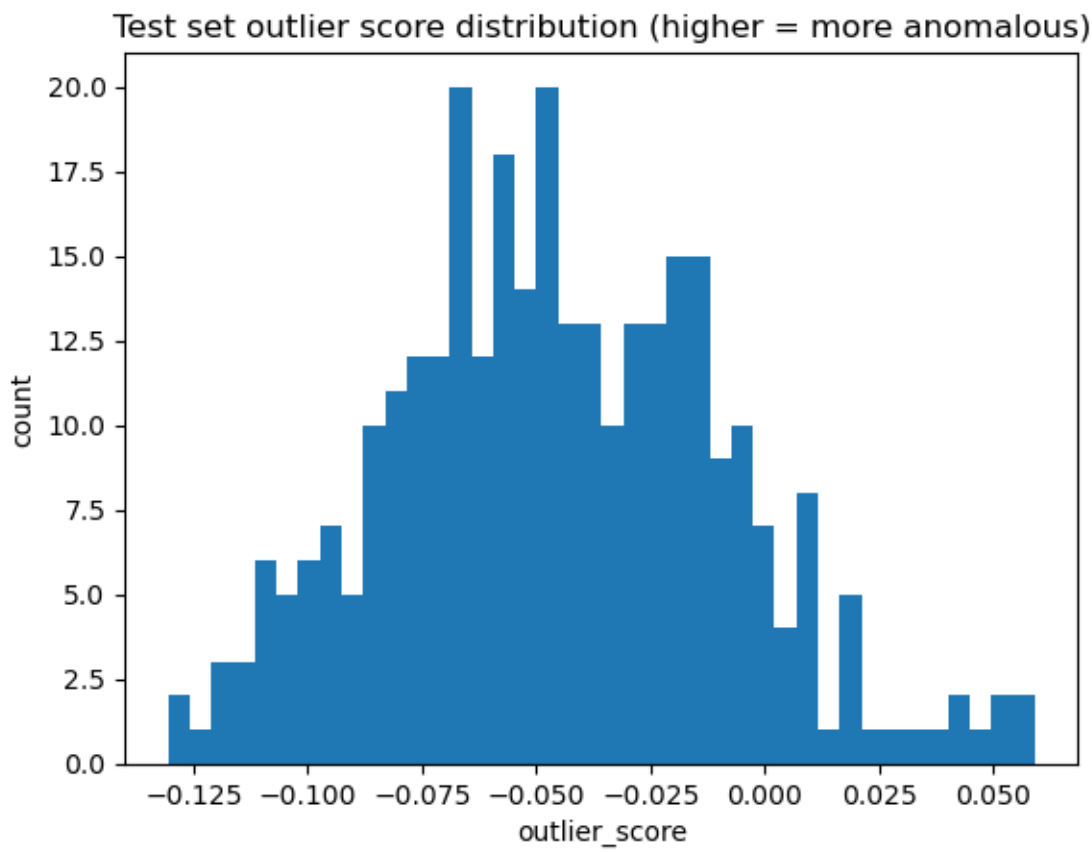
[34]:
```
pred_train_raw = pipe.predict(X_train)
pred_test_raw = pipe.predict(X_test)

train_outlier = (pred_train_raw == -1).astype(int)
test_outlier = (pred_test_raw == -1).astype(int)
```

[35]:
```
print("Predicted outlier rate (train):", train_outlier.mean())
print("Predicted outlier rate (test): ", test_outlier.mean())
```

```
Predicted outlier rate (train): 0.08082191780821918
Predicted outlier rate (test):  0.09872611464968153
```

[36]:
```
# decision_function: higher = more normal; invert to get "outlier score"
train_outlier_score = -pipe.decision_function(X_train)
test_outlier_score = -pipe.decision_function(X_test)
```

[37]:
```
plt.figure()
plt.hist(test_outlier_score, bins=40)
plt.title("Test set outlier score distribution (higher = more anomalous)")
plt.xlabel("outlier_score")
plt.ylabel("count")
plt.show()
```



[38]:
```
# Interpretation
k = 10
test_idx = np.argsort(test_outlier_score)[-k:][::-1]
top_outliers = X_test.iloc[test_idx].copy()
top_outliers["outlier_score"] = test_outlier_score[test_idx]
top_outliers["pred_outlier"] = test_outlier[test_idx]

print("Top predicted outliers (test):")
display(top_outliers.T)
```

```
Top predicted outliers (test):

                   882       247       336         829       371       800  \
school              MS        GP        GP          MS        MS        GP
sex                  F         M         F           F         M         F
address              R         U         R           U         R         U
famsize            LE3       GT3       GT3         LE3       LE3       LE3
Pstatus              A         T         A           A         T         A
Mjob             other  services  services     at_home   at_home   at_home
Fjob             other  services   at_home       other  services     other
reason          course     other      home  reputation     other      home
```

guardian	other	mother	other	mother	father	other
schoolsup	no	no	no	no	no	no
famsup	yes	no	no	no	yes	no
paid	no	no	yes	no	yes	no
activities	no	no	no	yes	yes	no
nursery	no	no	yes	no	yes	yes
higher	no	no	yes	no	no	no
internet	no	yes	no	no	yes	yes
romantic	yes	yes	no	yes	yes	no
subject	por	mat	mat	por	mat	por
age	18	22	19	16	18	19
Medu	3	3	3	2	1	2
Fedu	2	1	1	2	2	3
traveltime	2	1	1	2	3	2
studytime	3	1	3	4	1	1
failures	2	3	1	0	0	1
famrel	3	5	5	1	4	2
freetime	3	4	4	2	3	2
goout	2	5	3	1	3	3
Dalc	1	5	1	1	2	3
Walc	1	5	2	1	3	4
health	2	1	5	1	3	5
absences	6	16	12	4	3	16
G3	10	8	13	11	12	11
outlier_score	0.059078	0.054556	0.052133	0.049877	0.045266	0.044513
pred_outlier	1	1	1	1	1	1

	808	310	361	370
school	GP	GP	MS	MS
sex	M	F	M	F
address	R	U	R	U
famsize	LE3	LE3	LE3	LE3
Pstatus	T	T	T	T
Mjob	at_home	services	at_home	services
Fjob	other	services	other	services
reason	course	home	other	home
guardian	other	other	mother	other
schoolsup	no	no	no	no
famsup	yes	no	no	no
paid	no	no	no	no
activities	yes	yes	yes	yes
nursery	yes	no	no	yes
higher	no	yes	no	yes
internet	yes	no	no	no
romantic	yes	yes	no	yes
subject	por	mat	mat	mat
age	21	19	18	19
Medu	1	1	1	3

```
Fedu                  1         2        1         2
traveltime            2         1        2         2
studytime             2         2        2         2
failures              2         1        1         2
famrel                5         4        4         3
freetime              3         2        4         2
goout                 3         4        3         2
Dalc                  5         2        2         1
Walc                  2         2        3         1
health                4         3        5         3
absences             21         0        2         4
G3                   10         0       12         9
outlier_score  0.040943  0.038493  0.03444  0.028556
pred_outlier          1         1        1         1
```

[39]:
```
test_with_flags = X_test.copy()
test_with_flags["pred_outlier"] = test_outlier
for col in [c for c in num_cols if c in test_with_flags.columns]:
    grp = test_with_flags.groupby("pred_outlier")[col].agg(["mean", "median"])
    print(f"\nNumeric feature summary by predicted outlier flag: {col}")
    display(grp)
```

```
Numeric feature summary by predicted outlier flag: age

                    mean  median
pred_outlier
0              16.611307    16.0
1              18.161290    18.0


Numeric feature summary by predicted outlier flag: Medu

                   mean  median
pred_outlier
0              2.558304     3.0
1              1.935484     2.0


Numeric feature summary by predicted outlier flag: Fedu

                   mean  median
pred_outlier
0              2.325088     2.0
1              1.870968     2.0


Numeric feature summary by predicted outlier flag: traveltime

                   mean  median
pred_outlier
```

```
0             1.512367     1.0
1             1.741935     2.0


Numeric feature summary by predicted outlier flag: studytime

                  mean  median
pred_outlier
0             2.060071     2.0
1             1.838710     2.0


Numeric feature summary by predicted outlier flag: failures

                  mean  median
pred_outlier
0             0.229682     0.0
1             1.064516     1.0


Numeric feature summary by predicted outlier flag: famrel

                  mean  median
pred_outlier
0             3.893993     4.0
1             3.806452     4.0


Numeric feature summary by predicted outlier flag: freetime

                  mean  median
pred_outlier
0             3.212014     3.0
1             3.225806     3.0


Numeric feature summary by predicted outlier flag: goout

                  mean  median
pred_outlier
0             3.120141     3.0
1             3.161290     3.0


Numeric feature summary by predicted outlier flag: Dalc

                  mean  median
pred_outlier
0             1.427562     1.0
1             2.000000     1.0


Numeric feature summary by predicted outlier flag: Walc
```

```
                   mean  median
pred_outlier
0              2.173145     2.0
1              2.483871     2.0


Numeric feature summary by predicted outlier flag: health

                   mean  median
pred_outlier
0              3.579505     4.0
1              3.322581     3.0


Numeric feature summary by predicted outlier flag: absences

                   mean  median
pred_outlier
0              4.106007     2.0
1              8.000000     6.0


Numeric feature summary by predicted outlier flag: G3

                    mean  median
pred_outlier
0              11.371025    11.0
1              10.129032    10.0
```

[40]:
```
df_test = X_test.copy()
df_test["outlier"] = test_outlier
df_test["outlier_score"] = test_outlier_score

normal = df_test[df_test["outlier"] == 0]
anomalies = df_test[df_test["outlier"] == 1]

normal_mean = normal[num_cols].mean()
anomaly_mean = anomalies[num_cols].mean()

feature_contribution = anomaly_mean - normal_mean

contribution_df = pd.DataFrame({
    "normal_mean": normal_mean,
    "anomaly_mean": anomaly_mean,
    "difference": feature_contribution
})

contribution_df = contribution_df.reindex(contribution_df["difference"].abs().
 ↪sort_values(ascending=False).index)
```

```
display(contribution_df)
```

```
            normal_mean  anomaly_mean  difference
absences       4.106007      8.000000    3.893993
age           16.611307     18.161290    1.549983
G3            11.371025     10.129032   -1.241992
failures       0.229682      1.064516    0.834834
Medu           2.558304      1.935484   -0.622820
Dalc           1.427562      2.000000    0.572438
Fedu           2.325088      1.870968   -0.454121
Walc           2.173145      2.483871    0.310726
health         3.579505      3.322581   -0.256925
traveltime     1.512367      1.741935    0.229568
studytime      2.060071      1.838710   -0.221361
famrel         3.893993      3.806452   -0.087541
goout          3.120141      3.161290    0.041149
freetime       3.212014      3.225806    0.013792
```

[41]:
```
# Sensitivity Analysis
def fit_iforest_get_outputs(
    X_train, X_test, preprocess,
    contamination=0.08, n_estimators=200, max_samples="auto",
    random_state=42
):
    pipe = Pipeline(
        steps=[
            ("prep", preprocess),
            ("iforest", IsolationForest(
                n_estimators=n_estimators,
                contamination=contamination,
                max_samples=max_samples,
                random_state=random_state,
                n_jobs=-1
            ))
        ]
    )
    pipe.fit(X_train)
    outlier_score = -pipe.decision_function(X_test)
    pred_outlier = (pipe.predict(X_test) == -1).astype(int)

    return pipe, outlier_score, pred_outlier

def topk_overlap(score_a, score_b, k=50):
    """Overlap fraction of the top-k highest outlier scores."""
    idx_a = set(np.argsort(score_a)[-k:])
    idx_b = set(np.argsort(score_b)[-k:])
    return len(idx_a & idx_b) / k
```

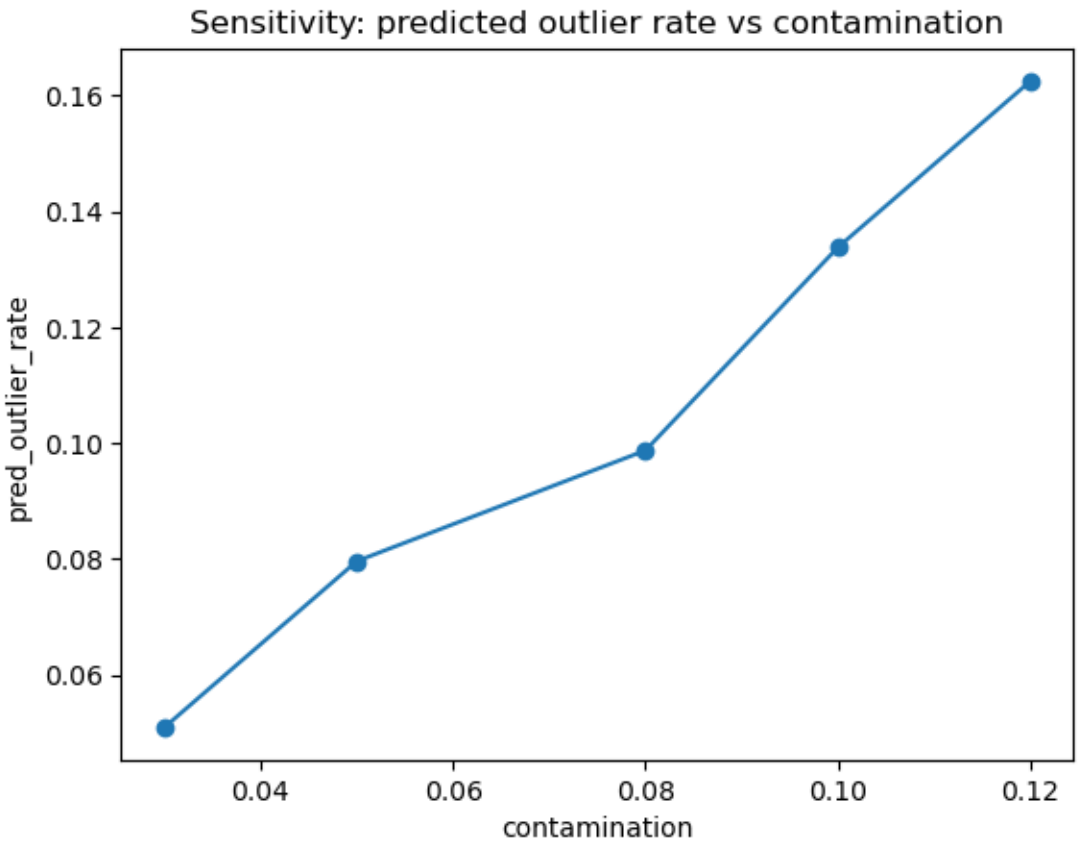
[42]:
```
BASE_CONTAM = base_contamination  # e.g., 0.08
BASE_TREES = 200
BASE_MAXS = "auto"
TOPK = 50

_, base_score, base_pred = fit_iforest_get_outputs(
    X_train, X_test, preprocess,
    contamination=BASE_CONTAM,
    n_estimators=BASE_TREES,
    max_samples=BASE_MAXS,
    random_state=RANDOM_STATE
)

print("Base predicted outlier rate:", float(base_pred.mean()))
```

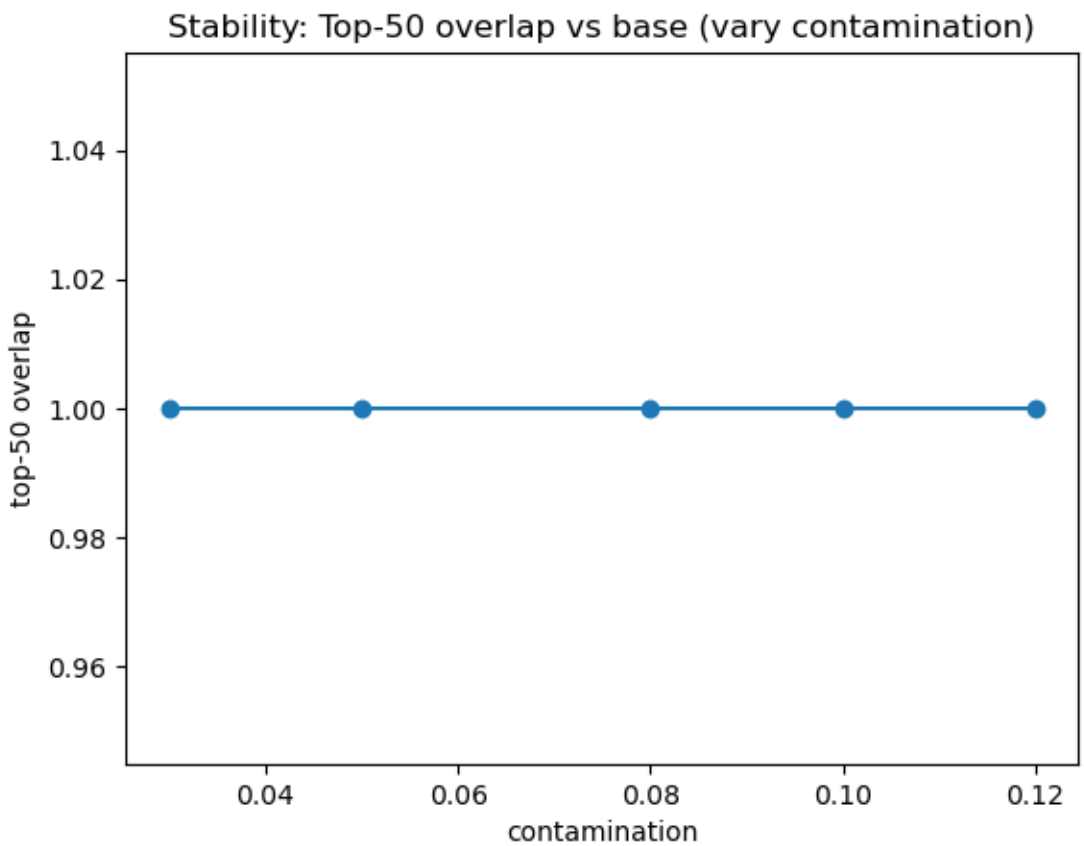
```
Base predicted outlier rate: 0.09872611464968153
```

[43]:
```
# Contamination
contamination_grid = [0.03, 0.05, 0.08, 0.10, 0.12]

rows = []
for c in contamination_grid:
    _, score, pred = fit_iforest_get_outputs(
        X_train, X_test, preprocess,
        contamination=c,
        n_estimators=BASE_TREES,
        max_samples=BASE_MAXS,
        random_state=RANDOM_STATE
    )
    rows.append({
        "contamination": c,
        "n_estimators": BASE_TREES,
        "max_samples": BASE_MAXS,
        "pred_outlier_rate": float(pred.mean()),
        "top{}_overlap_vs_base".format(TOPK): topk_overlap(base_score, score,␣
 ↪k=TOPK),
        "score_mean": float(np.mean(score)),
        "score_std": float(np.std(score)),
    })

sens_contam_df = pd.DataFrame(rows).sort_values("contamination")
print("Sensitivity: contamination")
display(sens_contam_df)

plt.figure()
plt.plot(sens_contam_df["contamination"], sens_contam_df["pred_outlier_rate"],␣
 ↪marker="o")
```

```
plt.title("Sensitivity: predicted outlier rate vs contamination")
plt.xlabel("contamination")
plt.ylabel("pred_outlier_rate")
plt.show()

plt.figure()
plt.plot(sens_contam_df["contamination"],␣
 ↪sens_contam_df["top{}_overlap_vs_base".format(TOPK)], marker="o")
plt.title(f"Stability: Top-{TOPK} overlap vs base (vary contamination)")
plt.xlabel("contamination")
plt.ylabel(f"top-{TOPK} overlap")
plt.show()
```

```
Sensitivity: contamination

   contamination  n_estimators max_samples  pred_outlier_rate  \
0           0.03           200        auto           0.050955
1           0.05           200        auto           0.079618
2           0.08           200        auto           0.098726
3           0.10           200        auto           0.133758
4           0.12           200        auto           0.162420

   top50_overlap_vs_base  score_mean  score_std
0                    1.0   -0.061775   0.036722
1                    1.0   -0.052483   0.036722
2                    1.0   -0.045489   0.036722
3                    1.0   -0.041564   0.036722
4                    1.0   -0.036893   0.036722
```


Sensitivity: predicted outlier rate vs contamination
pred_outlier_rate
0.06
0.08
0.10
0.12
0.14
0.16
0.04
0.06
0.08
0.10
0.12
contamination

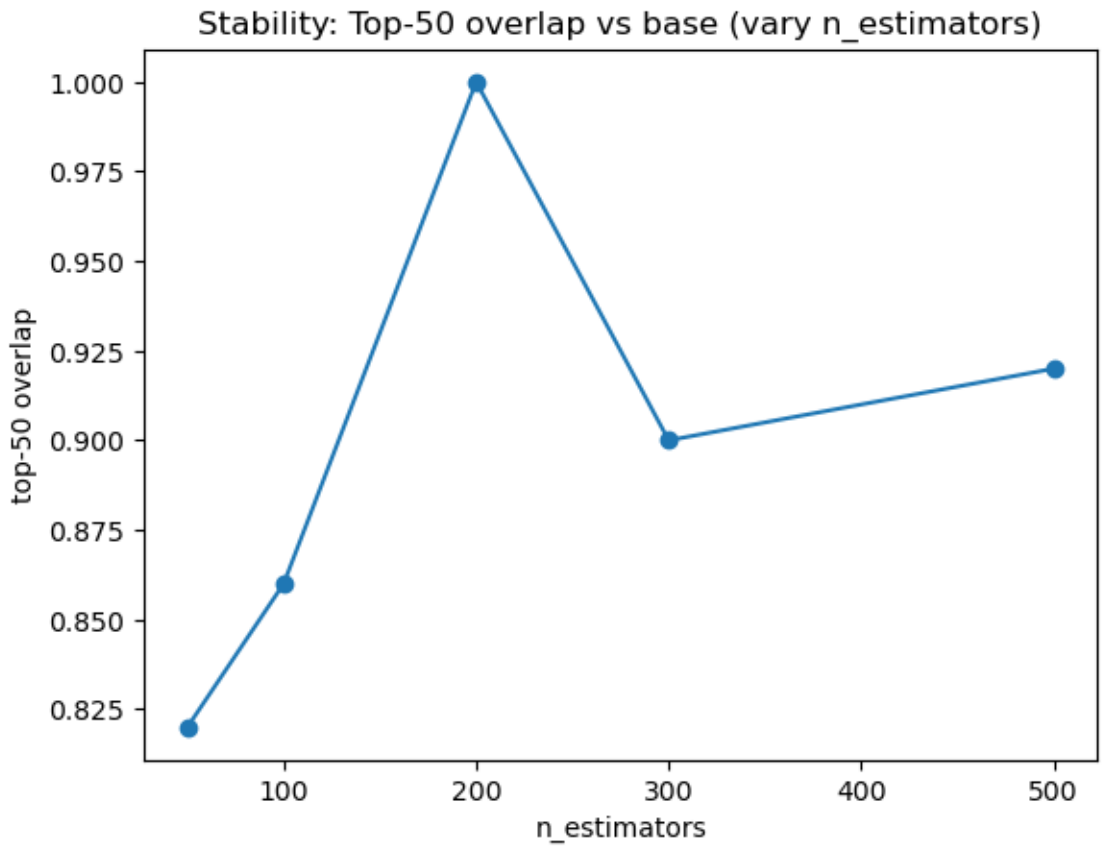


As the contamination parameter increases, the model flags more students as outliers. However, the overlap of the top 50 anomaly scores remains 1.0 across all settings, meaning the same students are consistently identified as the most abnormal. This suggests the anomaly detection results are stable and not highly sensitive to the contamination parameter.

[44]:
```
# N_estimators
n_estimators_grid = [50, 100, 200, 300, 500]

rows = []
for n in n_estimators_grid:
    _, score, pred = fit_iforest_get_outputs(
        X_train, X_test, preprocess,
        contamination=BASE_CONTAM,
        n_estimators=n,
        max_samples=BASE_MAXS,
        random_state=RANDOM_STATE
    )
    rows.append({
        "contamination": BASE_CONTAM,
        "n_estimators": n,
```

```
        "max_samples": BASE_MAXS,
        "pred_outlier_rate": float(pred.mean()),
        "top{}_overlap_vs_base".format(TOPK): topk_overlap(base_score, score,␣
 ↪k=TOPK),
        "score_mean": float(np.mean(score)),
        "score_std": float(np.std(score)),
    })

sens_trees_df = pd.DataFrame(rows).sort_values("n_estimators")
print("\nSensitivity: n_estimators")
display(sens_trees_df)

plt.figure()
plt.plot(sens_trees_df["n_estimators"], sens_trees_df["top{}_overlap_vs_base".
 ↪format(TOPK)], marker="o")
plt.title(f"Stability: Top-{TOPK} overlap vs base (vary n_estimators)")
plt.xlabel("n_estimators")
plt.ylabel(f"top-{TOPK} overlap")
plt.show()
```

```
Sensitivity: n_estimators

   contamination  n_estimators max_samples  pred_outlier_rate  \
0           0.08            50        auto           0.117834
1           0.08           100        auto           0.101911
2           0.08           200        auto           0.098726
3           0.08           300        auto           0.111465
4           0.08           500        auto           0.114650

   top50_overlap_vs_base  score_mean  score_std
0                   0.82   -0.051131   0.041279
1                   0.86   -0.046323   0.037936
2                   1.00   -0.045489   0.036722
3                   0.90   -0.044973   0.036398
4                   0.92   -0.045490   0.036411
```

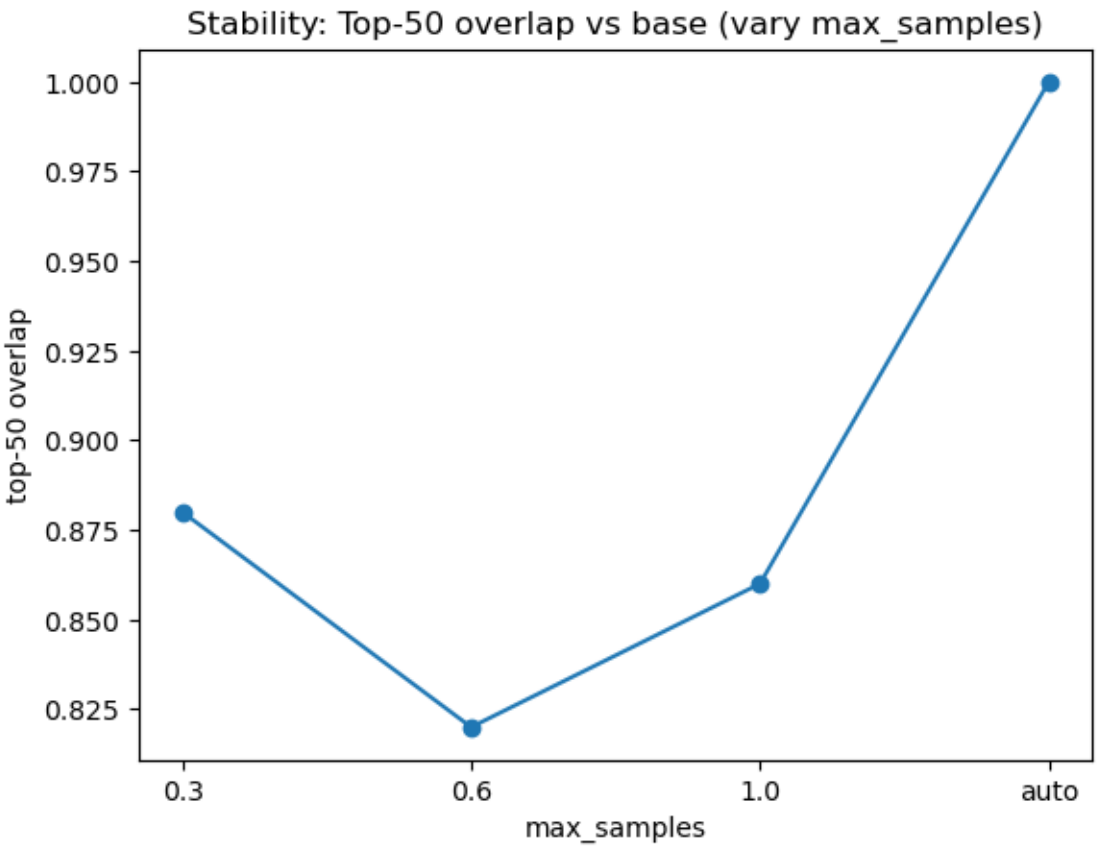


The sensitivity analysis shows that the anomaly detection results become more stable as the number of trees increases. With only 50 trees, the overlap with the base model is about 0.82, indicating some variation in detected anomalies.

[45]:
```
max_samples_grid = [0.3, 0.6, 1.0, "auto"]

rows = []
for ms in max_samples_grid:
    _, score, pred = fit_iforest_get_outputs(
        X_train, X_test, preprocess,
        contamination=BASE_CONTAM,
        n_estimators=BASE_TREES,
        max_samples=ms,
        random_state=RANDOM_STATE
    )
    rows.append({
        "contamination": BASE_CONTAM,
        "n_estimators": BASE_TREES,
        "max_samples": ms,
        "pred_outlier_rate": float(pred.mean()),
```

```
        "top{}_overlap_vs_base".format(TOPK): topk_overlap(base_score, score,␣
 ↪k=TOPK),
        "score_mean": float(np.mean(score)),
        "score_std": float(np.std(score)),
    })

sens_samples_df = pd.DataFrame(rows)
print("\nSensitivity: max_samples")
display(sens_samples_df)

plt.figure()
plt.plot(sens_samples_df["max_samples"].astype(str),␣
 ↪sens_samples_df["top{}_overlap_vs_base".format(TOPK)], marker="o")
plt.title(f"Stability: Top-{TOPK} overlap vs base (vary max_samples)")
plt.xlabel("max_samples")
plt.ylabel(f"top-{TOPK} overlap")
plt.show()
```

```
Sensitivity: max_samples

   contamination  n_estimators max_samples  pred_outlier_rate  \
0           0.08           200         0.3           0.101911
1           0.08           200         0.6           0.121019
2           0.08           200         1.0           0.133758
3           0.08           200        auto           0.098726

   top50_overlap_vs_base  score_mean  score_std
0                   0.88   -0.049357   0.038143
1                   0.82   -0.045919   0.037922
2                   0.86   -0.044426   0.039318
3                   1.00   -0.045489   0.036722
```



Changing max_samples slightly affects the results. As it increases from 0.3 to 1.0, the predicted outlier rate rises from about 10% to 13%, meaning more students are flagged as anomalies. The top-50 overlap stays between 0.82 and 0.88, showing most detected anomalies remain similar.

1.9 Demo Website

To demonstrate the practical use of our anomaly detection model, we developed an interactive web application using **Streamlit**. The goal of this demo is to allow users to explore how different student behavior patterns affect anomaly detection results.

Note: The demo is implemented as a Python script and must be executed from a `.py` file using Streamlit. To launch the application locally, run:

python train_model.py (first part, whatever you named it)

Then, run:

streamlit run app.py (second part, whatever you named it)

1.9.1 1. Model Training for Demo Website

[ ]:
```
import numpy as np
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import IsolationForest


RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)


# 1 Load datasets

mat_path = "student/student-mat.csv"
por_path = "student/student-por.csv"

df_mat = pd.read_csv(mat_path, sep=";")
df_mat["subject"] = "mat"

df_por = pd.read_csv(por_path, sep=";")
df_por["subject"] = "por"

df = pd.concat([df_mat, df_por], axis=0, ignore_index=True)

print("Dataset shape:", df.shape)

# 2 Remove leakage variables

DROP_COLS = ["G1", "G2"]

df_model = df.drop(columns=DROP_COLS).copy()
df_model = df_model.drop_duplicates().reset_index(drop=True)

print("After dropping G1/G2:", df_model.shape)

# ---------------------------
# 3 Identify feature types
# ---------------------------

cat_cols = df_model.select_dtypes(include=["object"]).columns.tolist()
num_cols = df_model.select_dtypes(include=[np.number]).columns.tolist()
```

```python
X = df_model[cat_cols + num_cols].copy()

print("Categorical features:", cat_cols)
print("Numeric features:", num_cols)

# ---------------------------
# 4 Preprocessing
# ---------------------------

preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ],
    remainder="drop",
)

# ---------------------------
# 5 Train/Test split
# ---------------------------

X_train, X_test = train_test_split(
    X,
    test_size=0.30,
    random_state=RANDOM_STATE
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)

# ---------------------------
# 6 Isolation Forest model
# ---------------------------

base_contamination = 0.08

pipe = Pipeline(
    steps=[
        ("prep", preprocess),
        ("iforest", IsolationForest(
            n_estimators=200,
            contamination=base_contamination,
            max_samples="auto",
            random_state=RANDOM_STATE,
            n_jobs=-1,
        ))
```

```
    ]
)

# ---------------------------
# 7 Fit model
# ---------------------------

pipe.fit(X_train)

print("Model training complete")

# ---------------------------
# 8 Evaluate anomaly rate
# ---------------------------

pred_train = pipe.predict(X_train)
pred_test = pipe.predict(X_test)

train_outlier = (pred_train == -1).astype(int)
test_outlier = (pred_test == -1).astype(int)

print("Predicted outlier rate (train):", train_outlier.mean())
print("Predicted outlier rate (test):", test_outlier.mean())

# ---------------------------
# 9 Save trained model
# ---------------------------

joblib.dump(pipe, "iforest_model.pkl")

print("Model saved as iforest_model.pkl")
```

This script is written as a Python training script (.py file) and should be executed from the command line to train the anomaly detection model. It builds an Isolation Forest pipeline using the student performance dataset. The math and Portuguese datasets are first combined, and intermediate grade variables (G1, G2) are removed to avoid information leakage. Numerical features are standardized and categorical features are one-hot encoded through a preprocessing pipeline. The processed data is then used to train the Isolation Forest model to identify unusual student behavior patterns. After training, the entire pipeline is saved as iforest_model.pkl, which is later loaded by the Streamlit demo application for interactive anomaly detection.

1.9.2 2. Interactive Demo Application (Streamlit)

[ ]:
```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
```

```python
import matplotlib.pyplot as plt

st.set_page_config(page_title="Student Anomaly Detection", layout="wide")

st.title("Student Performance Anomaly Detection")

st.write(
"""
Interactive demo using **Isolation Forest** to detect anomalous student profiles
based on behavioral and academic attributes.
"""
)

# Load model

@st.cache_resource
def load_model():
    return joblib.load("iforest_model.pkl")

model = load_model()

# Load dataset

@st.cache_data
def load_data():

    mat = pd.read_csv("student/student-mat.csv", sep=";")
    mat["subject"] = "math"

    por = pd.read_csv("student/student-por.csv", sep=";")
    por["subject"] = "portuguese"

    df = pd.concat([mat, por], ignore_index=True)

    df = df.drop(columns=["G1", "G2"])

    return df

df = load_data()

# Dataset overview

st.header("Dataset Overview")

col1, col2, col3 = st.columns(3)

col1.metric("Total Students", len(df))
```

```
col2.metric("Features", df.shape[1])
col3.metric("Model", "Isolation Forest")

st.dataframe(df.head())

# Model prediction on dataset

X = df.copy()

scores = -model.decision_function(X)
preds = model.predict(X)

df["anomaly_score"] = scores
df["anomaly"] = preds

# Score distribution

st.header("Anomaly Score Distribution")

fig, ax = plt.subplots()

ax.hist(scores, bins=40)

ax.set_xlabel("Anomaly Score")
ax.set_ylabel("Frequency")
ax.set_title("Distribution of Anomaly Scores")

st.pyplot(fig)

# Top anomalies

st.header("Most Anomalous Students")

top_anomalies = df.sort_values("anomaly_score", ascending=False).head(10)

st.dataframe(top_anomalies)

# Interactive student check

st.header("Interactive Student Check")

st.write("Modify key attributes and detect whether the student profile is␣
 ↪anomalous.")

age = st.slider("Age", 15, 22, 17)
studytime = st.slider("Study time", 1, 4, 2)
failures = st.slider("Past failures", 0, 4, 0)
```

```python
absences = st.slider("Absences", 0, 50, 5)
goout = st.slider("Going out frequency", 1, 5, 3)
freetime = st.slider("Free time", 1, 5, 3)

if st.button("Detect anomaly"):

    # Use template student
    sample = df.iloc[[0]].copy()

    sample["age"] = age
    sample["studytime"] = studytime
    sample["failures"] = failures
    sample["absences"] = absences
    sample["goout"] = goout
    sample["freetime"] = freetime

    score = -model.decision_function(sample)[0]
    pred = model.predict(sample)[0]

    st.subheader("Prediction Result")

    st.write("Anomaly score:", round(score, 4))

    if pred == -1:
        st.error("This student profile appears anomalous.")
    else:
        st.success("This student profile looks normal.")


# Scatter visualization

st.header("Absences vs Failures (Anomaly Visualization)")

fig2, ax2 = plt.subplots()

size = df["anomaly_score"] * 200

scatter = ax2.scatter(
    df["absences"],
    df["failures"],
    s=size,
    c=df["anomaly_score"],
    cmap="coolwarm",
    alpha=0.6
)

plt.colorbar(scatter, label="Anomaly Score")
```

```
ax2.set_xlabel("Absences")
ax2.set_ylabel("Failures")
ax2.set_title("Student Anomaly Detection")

st.pyplot(fig2)


# Behavior comparison

st.header("Behavior Pattern Comparison: Normal vs Anomaly")

features = [
    "age",
    "studytime",
    "failures",
    "absences",
    "goout",
    "freetime",
    "health",
    "Dalc",
    "Walc",
    "Medu"
]

normal = df[df["anomaly"] == 1]
anomaly = df[df["anomaly"] == -1]

normal_mean = normal[features].mean()
anomaly_mean = anomaly[features].mean()

fig3, ax3 = plt.subplots()

x = np.arange(len(features))
width = 0.35

ax3.bar(x - width/2, normal_mean, width, label="Normal")
ax3.bar(x + width/2, anomaly_mean, width, label="Anomaly")

ax3.set_xticks(x)
ax3.set_xticklabels(features, rotation=45)

ax3.set_ylabel("Average Value")
ax3.set_title("Behavioral Feature Comparison")

ax3.legend()
```

```
st.pyplot(fig3)
```

This script implements an interactive web application using Streamlit to demonstrate the trained Isolation Forest anomaly detection model. The application first loads the trained model (iforest_model.pkl) and the student performance datasets, combining the math and Portuguese data while removing the intermediate grade variables (G1, G2) to remain consistent with the training pipeline. The model is then applied to the dataset to compute anomaly scores and anomaly labels for each student, which are used to generate visualizations such as the anomaly score distribution and the most anomalous observations. The app also includes an interactive section where users can modify several behavioral variables—such as age, study time, past failures, absences, going-out frequency, and free time—to simulate a student profile. The modified profile is passed through the trained model to obtain an anomaly score and classification, allowing users to observe how different behavioral patterns influence anomaly detection.

1.9.3 Below is the PDF for the demo website:

Open the demo website PDF

Student Performance Anomaly Detection

Interactive demo using **Isolation Forest** to detect anomalous student profiles based on behavioral and academic attributes.

Dataset Overview

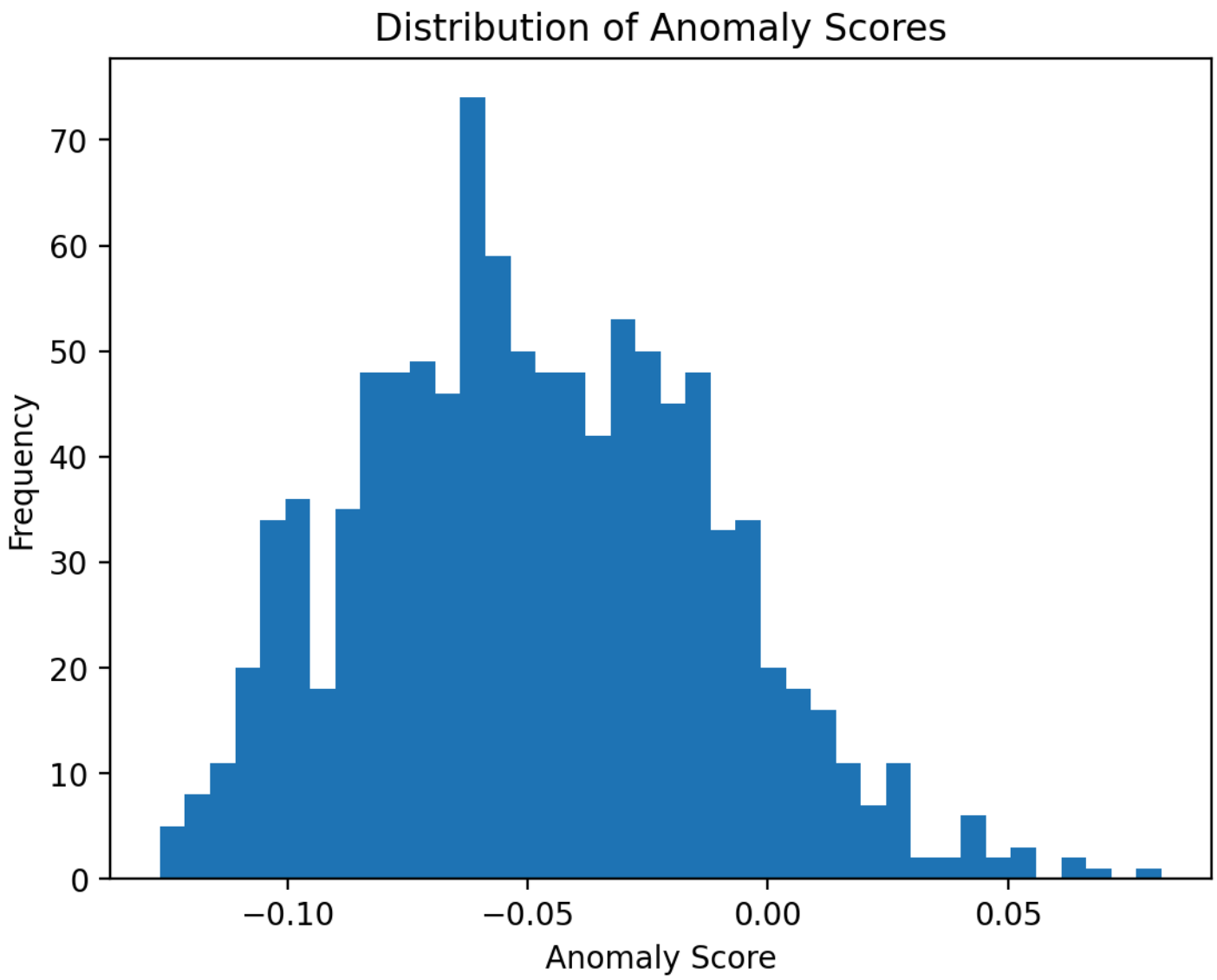
Total Students
1044

Features
32

Model
Isolation Forest

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason	guardian	traveltime	studytime	failures	schoolsup	famsup
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	course	mother	2	2	0	yes	no
1	GP	F	17	U	GT3	T	1	1	at_home	other	course	father	1	2	0	no	yes
2	GP	F	15	U	LE3	T	1	1	at_home	other	other	mother	1	2	3	yes	no
3	GP	F	15	U	GT3	T	4	2	health	services	home	mother	1	3	0	no	yes
4	GP	F	16	U	GT3	T	3	3	other	other	home	father	1	2	0	no	yes

Anomaly Score Distribution



Most Anomalous Students

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason	guardian	traveltime	studytime	failures	schoolsup	fams
1005	MS	F	19	R	GT3	A	1	1	at_home	at_home	course	other	2	2	3	no	yes
390	MS	M	20	U	LE3	A	2	2	services	services	course	other	1	2	2	no	yes
882	MS	F	18	R	LE3	A	3	2	other	other	course	other	2	3	2	no	yes
980	MS	F	17	R	LE3	A	3	1	other	at_home	course	other	2	3	0	no	yes
674	GP	M	22	U	GT3	T	3	1	services	services	other	mother	1	1	3	no	no
247	GP	M	22	U	GT3	T	3	1	services	services	other	mother	1	1	3	no	no
829	MS	F	16	U	LE3	A	2	2	at_home	other	reputation	mother	2	4	0	no	no
336	GP	F	19	R	GT3	A	3	1	services	at_home	home	other	1	3	1	no	no
808	GP	M	21	R	LE3	T	1	1	at_home	other	course	other	2	2	2	no	yes
1000	MS	F	19	U	GT3	T	1	1	at_home	services	other	father	2	1	1	no	no

Interactive Student Check

Modify key attributes and detect whether the student profile is anomalous.

Age

18

Study time

2

Past failures

1

Absences

5

Going out frequency

3

Free time

3

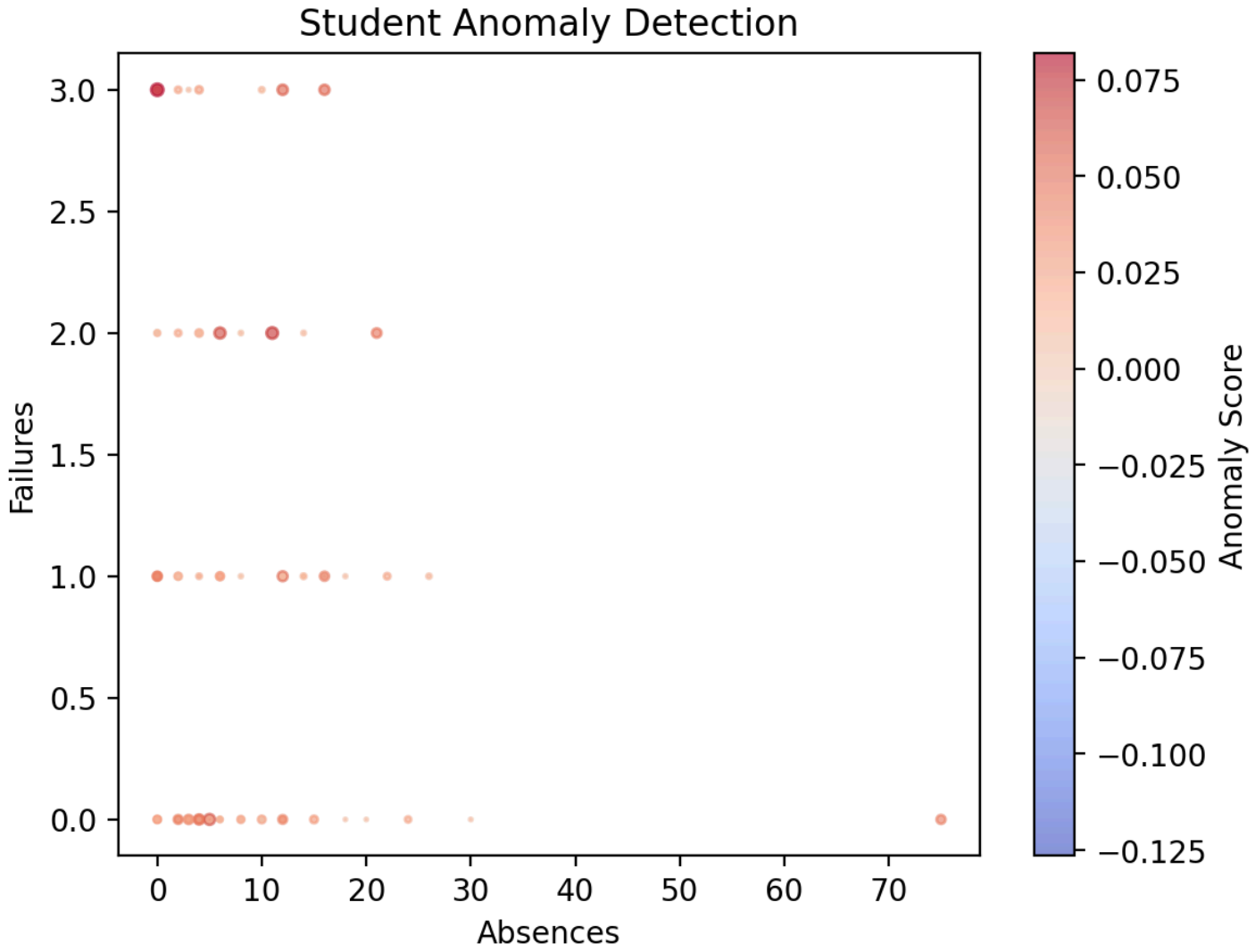
Detect anomaly

Prediction Result

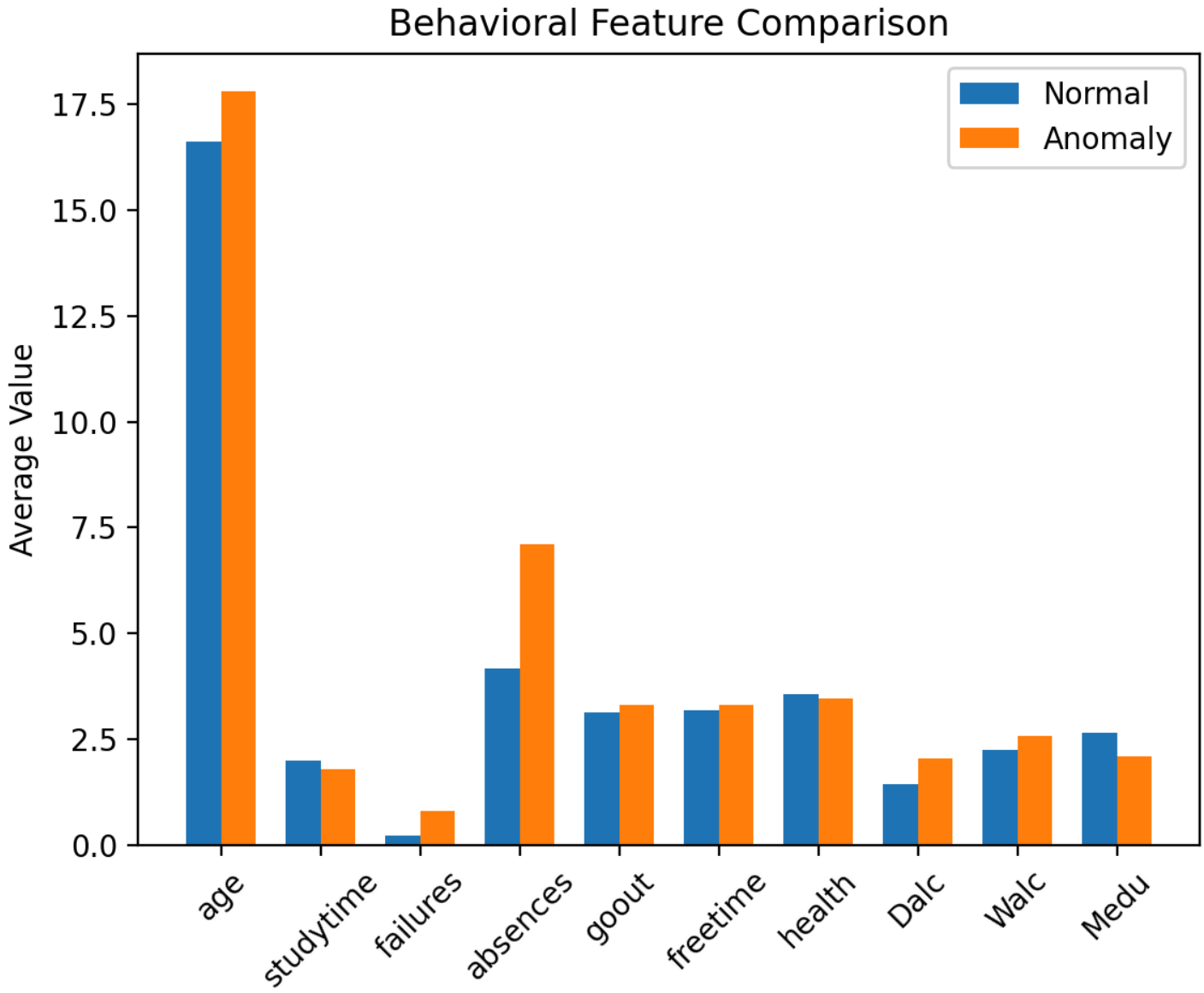
Anomaly score: `0.005`

This student profile appears anomalous.

Absences vs Failures (Anomaly Visualization)



Behavior Pattern Comparison: Normal vs Anomaly


Behavioral Feature Comparison
Normal
Anomaly
Average Value
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
age
studytime
failures
absences
goout
freetime
health
Dalc
Walc
Medu